FACULTY OF ENGINEERING AND TECHNOLOGY

ELECTRICAL AND COMPUTER ENGINEERING

DEPARTMENT

ADVANCED DIGITAL DESIGN System - ENCS3310

First Semester (2026 / 2025)

Project Report

Smart Traffic Light Controller with Pedestrian Request

Prepared by : Reem Taleb **ID:** 1232207

Instructor : Dr. Abdallatif Abuissa

Section : 3

Submission Date : January 8, 2026

State Diagram:

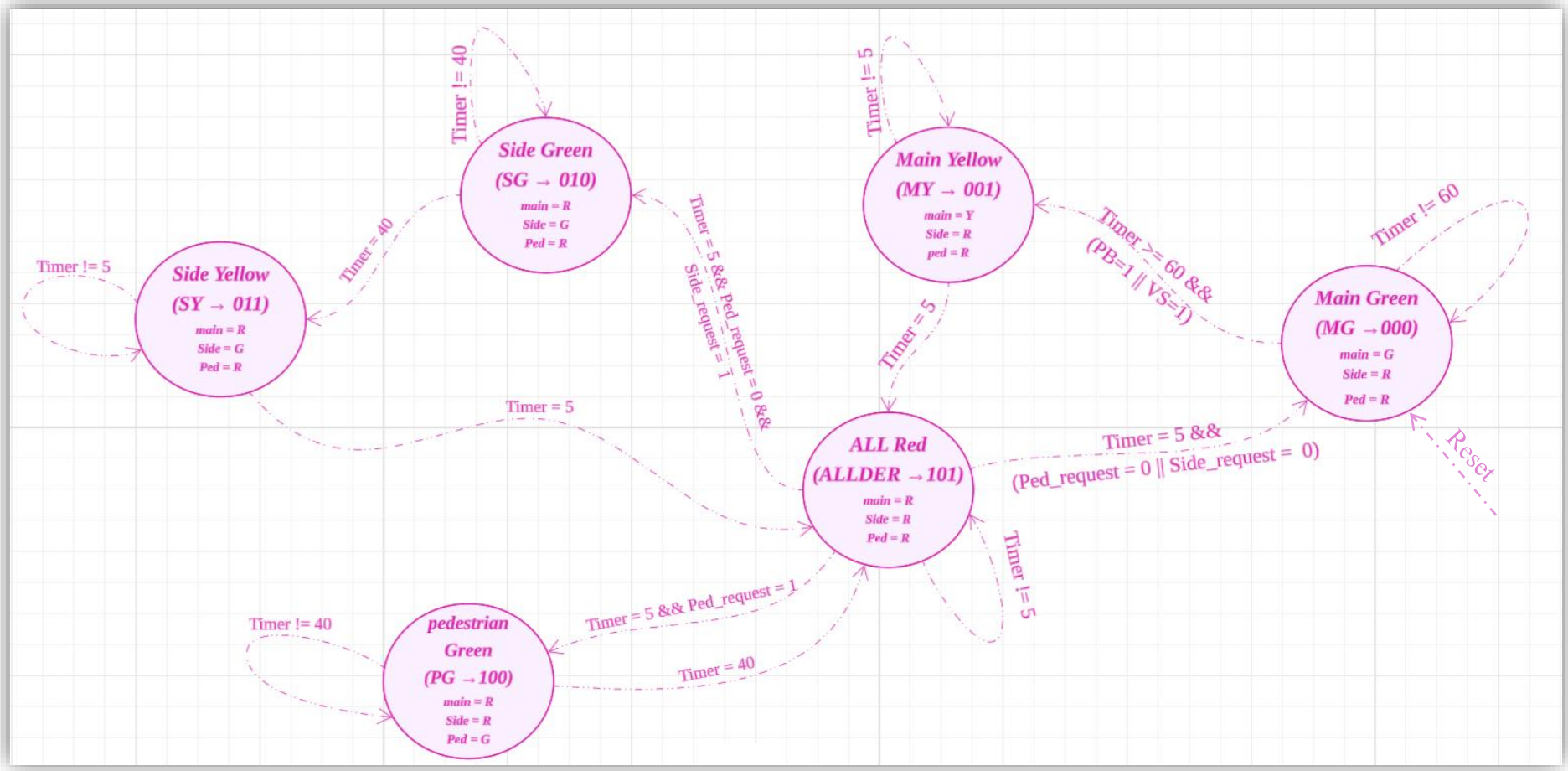

Figure 1 :State Diagram

The **State Diagram** represents the main structure that ensures the correct operation of the traffic light controller. It defines all possible system states and clearly shows how the controller moves between them based on time conditions and input request.The system starts in the **Main Green (MG)** state,which also serves as the **reset state**.In this state, the main road traffic light remains green for **60 seconesd**.After this time,the controller checks the input signals:the **pedestrain (PB)** & the **side road (VS)**, to decide whether the **Main state** should change to **Yellow**.

In case of any request, the system moves from **Main Green** to **Main Yellow (MY)** for **5 seconeds**, and then transition to the **ALL Red** state.The ALL Red state is a key reference point in the design, as it temporarily stopes all traffic and povides an organized moment decide which direction should be served next. From the ALL Red state, **Pedestrain request** are given **higher priority**.If a pedestrain request is active, the system transition to **Pedestrain Green (PG)**. If there is no pedestrain request and a side road request exisits, the system transtions to **Side Green (SG)**.If no request are present,the controller returns to the **Main Green** state.

To handle simultaneous or overlapping requests, the system uses an internal queuing. Any request received during an active green state is stored in a pending flag (queue). The request is not handled immediately; instead, it is served in the next control cycle after the system returns to the **ALL Red** state. This approach ensures orderly operation, preserves pedestrian priority, and prevents conflicting transitions.

Binary encoding is used to represent the FSM states, where each state has a unique binary value. State transitions occur only at the **rising edge** of the clock (posedge clk), which ensures synchronous behavior. Since state changes do not depend on unstable signals and occur only on clock edges, the design prevents **glitches** and **race conditions**. In addition, the system outputs depend only on the current state; therefor, this controller is designed as a **Moore machine**, not a Mealy machine. The output signals (traffic lights) are determined by the current state and are shown inside the state circles in the state diagram. The input signals do not directly affect the outputs; instead, they only influence the transition from one state to another. This design choice improves system stability and avoids unwanted output changes caused by sudden input .

Design Methodolgy:

The FSM states are implemented using binary encoding, as defined in the state diagram. A single 8-bit timer is used to control the duration of all FSM states. The timer width was selected to safely cover the maximum required timing value, which reaches up to 60 seconds. This timer is shared across all states and is reset whenever the controller transitions to a new state. This behavior is verified through simulation in the testbench.

For verification purposes, an additional signal (**verf_state**) is included to indicate the current FSM state during simulation. This signal is used for observation and debugging and does not affect the functional behavior of the system.

To manage asynchronous input requests, internal request flags (**Ped_request** and **Side_request**) are used. These flags act as a simple request-queuing mechanism, ensuring that short or overlapping input pulses are not lost. If a request occurs while the controller is in a green state, it is stored and serviced later. Pedestrian requests are given higher priority than side-road requests.

When the timer expires in the Main Green state, the controller checks whether any request is pending. Pedestrian requests are evaluated first, followed by side-road requests. Upon detecting a valid request, the system transitions to the Main Yellow state, resets the timer, and remains there for 5 seconds before entering the ALL-Red state.

The ALL-Red state acts as a decision state in which the next green phase is selected based on the stored requests. If a pedestrian request exists, the controller transitions to the Pedestrian Green state; otherwise, it transitions to the Side Green state. Each green state remains active for 40 seconds before transitioning through its corresponding yellow state and returning to the ALL-Red state.

Handling Special and Error Scenarios:

Verification & Testing Methodology:

- Reference module:

The **traffic_ctrl_ref** module is a **reference module** designed behaviorally to **verify the outputs of the original RTL module**. Verification was performed using a self-checking testbench.It takes the current traffic state (mode) as input and generates the expected outputs: the main street lights (**light_main**), the side street lights (**light_side**), and the pedestrian signal (**walk**).

The module is meant **only** for **verification**, treating the **RTL** as a **black box**. During simulation, the outputs of the RTL design were continuously compared against the reference model. Any mismatch is automatically reported, allowing immediate detection of functional errors.

- TestBench:

This testbench is designed to fully verify the RTL traffic controller using a reference (verification) module. All **inputs** are defined as **reg** , while **outputs** are defined as **wire**. Additional registers are introduced to support verification, such as TC_State, prev_VS, prev_PB, prev_state, timing parameters (MGT, SGT, PGT, YT, ART) are defined using parameter statements to represent the required durations for each traffic state. These parameters allow the testbench to remain flexible and consistent with the project specifications. Both the RTL module and the reference module are instantiated. The RTL module outputs its internal verification state (**verf_state**), which is treated as a black-box state representation and passed directly to the reference module. This enables direct comparison between the RTL outputs and the expected outputs generated behaviorally by the reference module. A clock signal is generated with a period of 10ns, ensuring consistent timing across all test scenarios. The testbench applies several test cases:

- Reset validation.
- Normal operation with no requests (main road green).
- Pedestrian-only request.
- Side-vehicle-only request.

Simultaneous pedestrian and side-vehicle requests, verifying priority handling where pedestrian requests are served first, followed by the side vehicle request after returning to the ALL RED state.

The repeat statements combined with #10 delays are used to allow sufficient clock cycles for the internal timers to count and state transitions to occur correctly.

For self-verification, previous values of the state, pedestrian button, and side vehicle request are stored. This allows:

- Printing messages only when requests are ensuring new.
- Detecting and printing state transitions without redundant messages
- Confirming that each state's timer reaches its expected value

All the required printing sentences are shown in figure 2.

State change messages are printed only when the current state differs from the previous one, reducing unnecessary repetition and making the simulation output easier to analyze.

Finally, on every positive clock edge, the outputs of the RTL module are compared against the outputs of the reference module. An error is reported only if a mismatch occurs, printing the current state and both RTL and reference outputs before stopping the simulation. This ensures that only real functional errors are flagged.

In summary, this testbench performs automatic, cycle-accurate verification of the RTL traffic controller by comparing it against a behavioral reference model, validating both functionality and timing behavior.

```
run
# KERNEL: Time = 55 | STATE CHANGE: xxx ---> 000 | Timer = 0 | States: Main = 001 Side = 100 Walk = 0
# KERNEL: Time = 655 | Check: Main Green timer reached 60 (Timer =  60)
# KERNEL: Time = 755 | Pedestrian Button PRESSED (PB = 1)
# KERNEL: Time = 775 | STATE CHANGE: 000 ---> 001 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Time = 825 | Check: Yellow timer reached 5 (Timer =   5)
# KERNEL: Time = 835 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 845 | STATE CHANGE: 101 ---> 100 | Timer = 0 | States: Main = 100 Side = 100 Walk = 1
# KERNEL: Time = 1245 | Check: Pedestrian Green timer reached 40 (Timer =  40)
# KERNEL: Time = 1255 | STATE CHANGE: 100 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 1265 | STATE CHANGE: 101 ---> 000 | Timer = 0 | States: Main = 001 Side = 100 Walk = 0
# KERNEL: Time = 1865 | Check: Main Green timer reached 60 (Timer =  60)
# KERNEL: Time = 1875 | Side Vehicle REQUEST (VS = 1)
# KERNEL: Time = 1895 | STATE CHANGE: 000 ---> 001 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Time = 1945 | Check: Yellow timer reached 5 (Timer =   5)
# KERNEL: Time = 1955 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 1965 | STATE CHANGE: 101 ---> 010 | Timer = 0 | States: Main = 100 Side = 001 Walk = 0
# KERNEL: Time = 2365 | Check: Side Green timer reached 40 (Timer =  40)
# KERNEL: Time = 2375 | STATE CHANGE: 010 ---> 011 | Timer = 0 | States: Main = 100 Side = 010 Walk = 0
# KERNEL: Time = 2435 | STATE CHANGE: 011 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 2445 | STATE CHANGE: 101 ---> 000 | Timer = 0 | States: Main = 001 Side = 100 Walk = 0
# KERNEL: Time = 2995 | Pedestrian Button PRESSED (PB = 1)
# KERNEL: Time = 2995 | Side Vehicle REQUEST (VS = 1)
# KERNEL: Time = 3045 | Check: Main Green timer reached 60 (Timer =  60)
# KERNEL: Time = 3055 | STATE CHANGE: 000 ---> 001 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Time = 3105 | Check: Yellow timer reached 5 (Timer =   5)
# KERNEL: Time = 3115 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 3125 | STATE CHANGE: 101 ---> 100 | Timer = 0 | States: Main = 100 Side = 100 Walk = 1
# KERNEL: Time = 3525 | Check: Pedestrian Green timer reached 40 (Timer =  40)
# KERNEL: Time = 3535 | STATE CHANGE: 100 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 3545 | STATE CHANGE: 101 ---> 010 | Timer = 0 | States: Main = 100 Side = 001 Walk = 0
# KERNEL: Time = 3945 | Check: Side Green timer reached 40 (Timer =  40)
# KERNEL: Time = 3955 | STATE CHANGE: 010 ---> 011 | Timer = 0 | States: Main = 100 Side = 010 Walk = 0
# KERNEL: Time = 4015 | STATE CHANGE: 011 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 4025 | STATE CHANGE: 101 ---> 000 | Timer = 0 | States: Main = 001 Side = 100 Walk = 0
# RUNTIME: Info: RUNTIME_0068 trafficLightController.v (245): $finish called.
```

Figure2 : Result of Test Bench

- Error Scenarios discovered by the Reference module:

1.

```
91          light_main = 3'b100; //RED
92          light_side = 3'b100; //RED
93          walk = 0;
94
95          case (State)
96              MG: light_main = 3'b010; //Green Main    //Case Error Scenario : light_main = 3'b010 (Yellow insted of Green)
97              MY: light_main = 3'b010; //Yellow Main
```

```
Console
# 3:50 AM, Friday, January 9, 2026
# Simulation has been initialized
run
# KERNEL: Time = 55 | STATE CHANGE: xxx ---> 000 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Error At Time = 55
# KERNEL: State = 000
# KERNEL: RTL : Main = 010 Side = 100 Walk = 0
# KERNEL: REF : Main = 001 Side = 100 Walk = 0
```

2.

```
106     PG: begin
107         light_main = 3'b100; //Red Main
108         light_side = 3'b100; //Red Side
109         walk = 0;            //Pedestrain Green    /*Case Error Scenario : walk = 0 Red Pedestrain insted of Green Pedestrain*/
```

```
Console
# KERNEL: Time = 55 | STATE CHANGE: xxx ---> 000 | Timer = 0 | States: Main = 001 Side = 100 Walk = 0
# KERNEL: Time = 655 | Check: Main Green timer reached 60
# KERNEL: Time = 755 | Pedestrian Button PRESSED (PB = 1)
# KERNEL: Time = 775 | STATE CHANGE: 000 ---> 001 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Time = 825 | Check: Yellow timer reached 5
# KERNEL: Time = 835 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 845 | STATE CHANGE: 101 ---> 100 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Error At Time = 845
# KERNEL: State = 100
# KERNEL: RTL : Main = 100 Side = 100 Walk = 0
# KERNEL: REF : Main = 100 Side = 100 Walk = 1
```

3.

```
98          SG: begin
99              light_main = 3'b001; //Red Main    /*Case Error Scenario : light_main = 3'b001 (Green insted of Red) cause a triffic accident*/
100             light_side = 3'b001; //Green Side
101         end
```

```
Console
# KERNEL: Time = 1955 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 100 Walk = 0
# KERNEL: Time = 1965 | STATE CHANGE: 101 ---> 010 | Timer = 0 | States: Main = 001 Side = 001 Walk = 0
# KERNEL: Error At Time = 1965
# KERNEL: State = 010
# KERNEL: RTL : Main = 001 Side = 001 Walk = 0
# KERNEL: REF : Main = 100 Side = 001 Walk = 0
```

4.

```
103     ALLRED: begin
104             light_main = 3'b100; //Red Main
105             light_side = 3'b001; //Red Side    /*Case Error Scenario : light_side = 3'b001 (Green light insted of Red light)*/
106             walk = 0;            // Pedestrain Red
```

```
Console
# KERNEL: Time = 755 | Pedestrian Button PRESSED (PB = 1)
# KERNEL: Time = 775 | STATE CHANGE: 000 ---> 001 | Timer = 0 | States: Main = 010 Side = 100 Walk = 0
# KERNEL: Time = 825 | Check: Yellow timer reached 5
# KERNEL: Time = 835 | STATE CHANGE: 001 ---> 101 | Timer = 0 | States: Main = 100 Side = 001 Walk = 0
# KERNEL: Error At Time = 835
# KERNEL: State = 101
# KERNEL: RTL : Main = 100 Side = 001 Walk = 0
# KERNEL: REF : Main = 100 Side = 100 Walk = 0
```

- Waveforms:

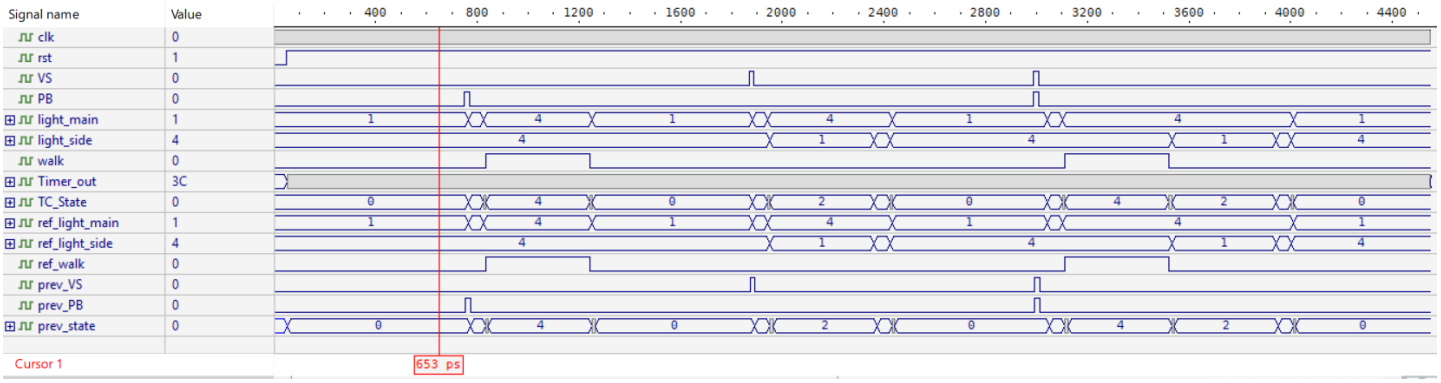

Figure3 : Waveform module

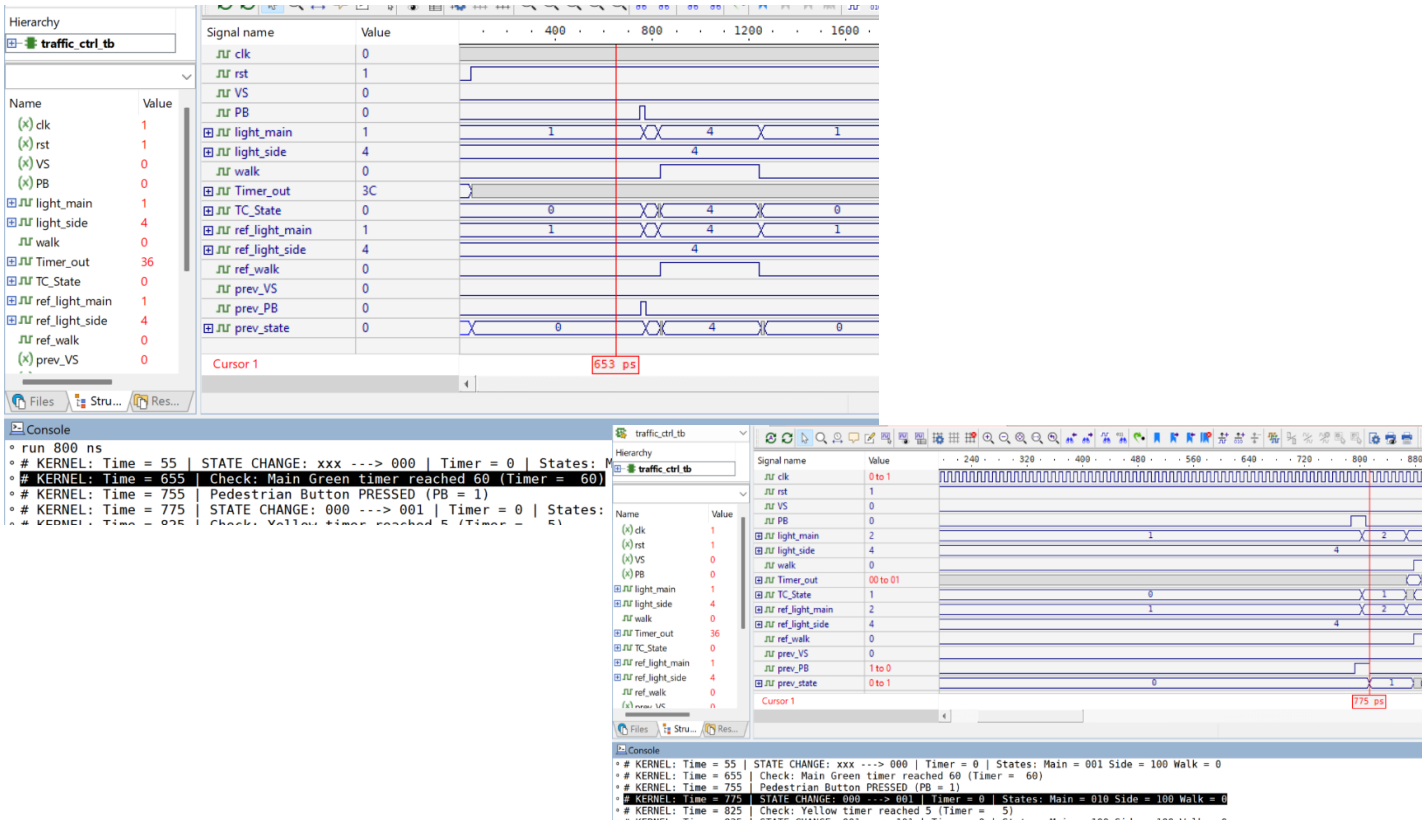

Figure4 :Cases for Waveform

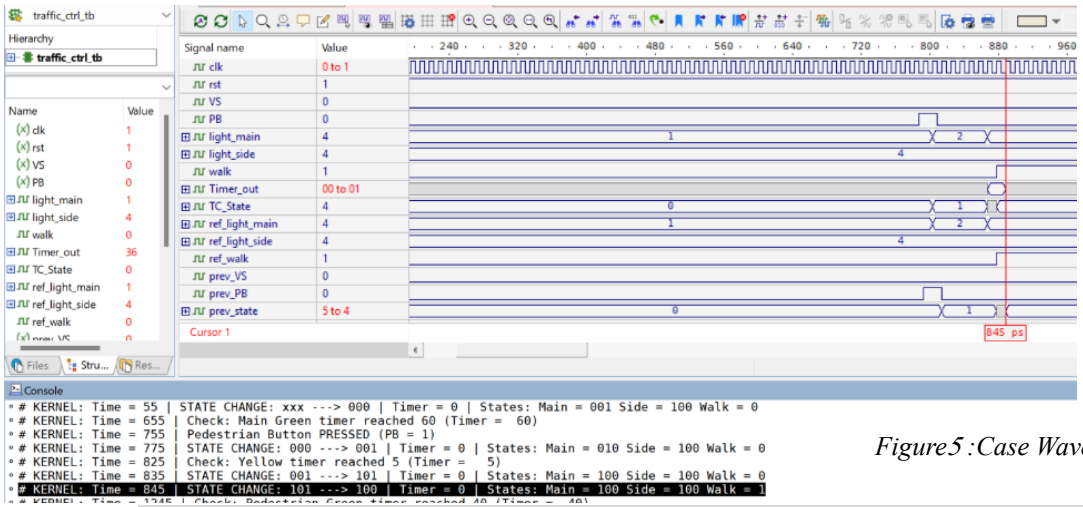

Figure5 :Case Waveform module

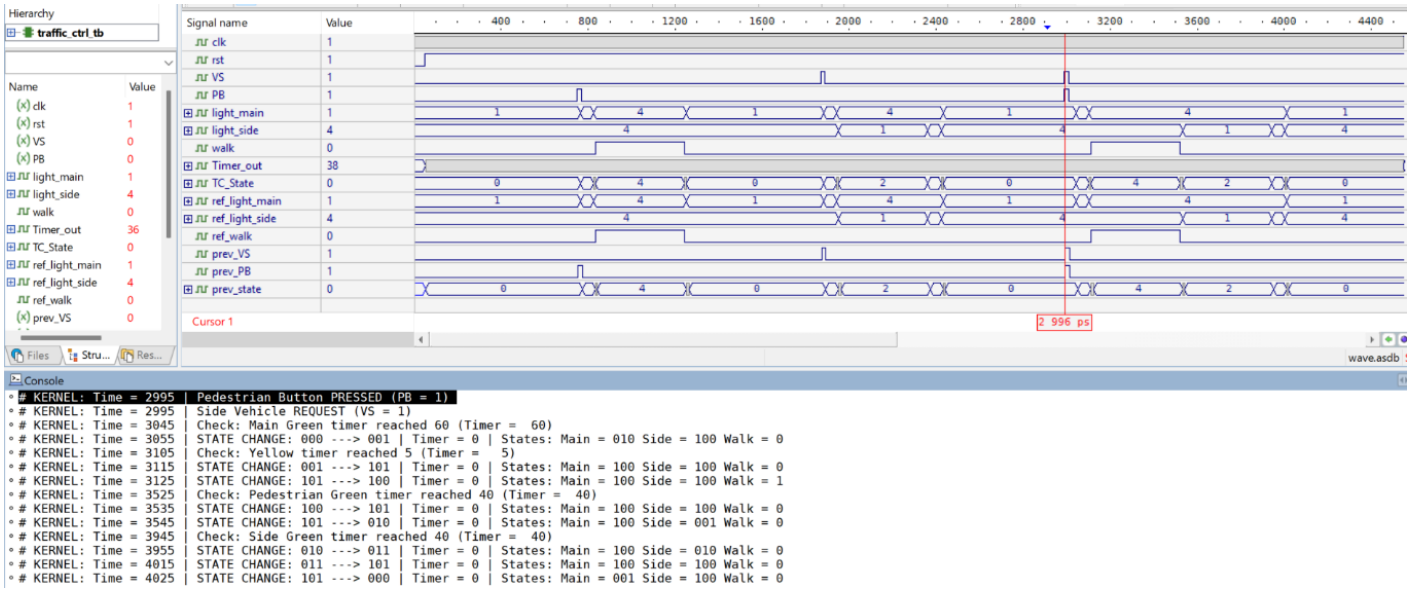

Figure6 :Cases Waveform module

As we can see, the waveform matches exactly with the testbench results, and all signals appear correctly and consistently, as shown in the figures. Looking at figure 3, we notice a clear agreement between the waveform and the values printed in the console. For example, as shown in figure 4, at **Time = 655**, the testbench values displayed in the console match the values shown in the waveform. At this moment, the **timer** reaches **3C**, which is equivalent to **60** in **decimal**. This corresponds to the **MG** (Main Green) state. Therefore, the **main road traffic light** should be **green**, while both the **pedestrian line** and the **side road** should be **red**. This behavior is clearly visible in the waveform. The **green traffic light** is represented by (**001**) in **binary**, which equals **1** in **decimal**, while the **red light for the side** road is represented by (**100**) in **binary**, which equals **4** in **decimal**. As for the **pedestrian line**, the signal remains **0** and **does not rise** to **1**, which perfectly matches the reference outputs, as shown in the figures. In figure 4 at **Time = 775**, the changes caused by the **state transition** from **MG** to **MY** are shown. This transition occurs due to the change in the **PB** (Pedestrian Button) input from **0** to **1** for a short period. As a result, the following changes occur:

- The **clock signal** changes from **0** to **1**.
- The **timer resets** to **zero** because the system moves to a new state.
- The **main road signal** changes to **2**, which corresponds to a **yellow light** (**010**), while the other traffic signals remain unchanged.

In figure 5 at **Time = 845**, the actual change of the **pedestrian traffic light occurs**. The **Walk signal** becomes **1** instead of **0**, and the **timer resets again** to start **counting up** to **40**, after which the **pedestrian signal turns off**. During this period, the **side road traffic light** remains **red** (**4**), and the **main road traffic light** changes from **yellow** to **red** (**4**). This happens because the system passes through the **ALL Red state** before transitioning to the **PG** state. All these results are exactly as expected, and we can clearly see the **agreement** between the **outputs** of **traffic_ctrl** and **traffic_ctrl_ref**, confirming the correct behavior of the system. Finally, figure 6 shows the state where both the **pedestrian signal** and the **side road signal** are **active** at the **same time**, following the same explanation and sequence described above.